This file is a merged representation of the entire codebase, combined into a single document by Repomix.

<file_summary>
This section contains a summary of this file.

<purpose>
This file contains a packed representation of the entire repository's contents.
It is designed to be easily consumable by AI systems for analysis, code review,
or other automated processes.
</purpose>

<file_format>
The content is organized as follows:
1. This summary section
2. Repository information
3. Directory structure
4. Repository files (if enabled)
4. Repository files, each consisting of:
  - File path as an attribute
  - Full contents of the file
</file_format>

<usage_guidelines>
- This file should be treated as read-only. Any changes should be made to the
  original repository files, not this packed version.
- When processing this file, use the file path to distinguish
  between different files in the repository.
- Be aware that this file may contain sensitive information. Handle it with
  the same level of security as you would the original repository.
</usage_guidelines>

<notes>
- Some files may have been excluded based on .gitignore rules and Repomix's configuration
- Binary files are not included in this packed representation. Please refer to the Repository Structure section for a complete list of file paths, including binary files
- Files matching patterns in .gitignore are excluded
- Files matching default ignore patterns are excluded
- Files are sorted by Git change count (files with more changes are at the bottom)
</notes>

<additional_info>

</additional_info>

</file_summary>

<directory_structure>
.gitignore
package.json
public/index.html

public/manifest.json
public/robots.txt
README.md
src/App.js
src/components/layout/AppLayout.jsx
src/components/privateRoute/PrivateRoute.jsx
src/components/publicRoute/PublicRoute.jsx
src/config.js
src/context/authContext.js
src/index.js
src/pages/accounts/AccountsPage.jsx
src/pages/accounts/ui/DepositWithdrawModal.jsx
src/pages/accounts/values/transactionTable.js
src/pages/credits/CreditsPage.jsx
src/pages/forbidden/ForbiddenPage.jsx
src/pages/home/HomePage.jsx
src/pages/login/Callback.jsx
src/pages/login/LoginPage.jsx
src/pages/notFound/NotFoundPage.jsx
src/pages/profile/ProfilePage.jsx
src/pages/tariffs/TariffsPage.jsx
src/pages/users/UsersPage.jsx
src/services/api.js
src/services/authService.js
src/services/core.js
src/services/credit.js
src/services/users.js
</directory_structure>

<files>
This section contains the contents of the repository's files.

<file path=".gitignore">
See https://help.github.com/articles/ignoring-files/ for more about ignoring
files.

dependencies
/node_modules
/.pnp
.pnp.js

testing
/coverage

production
/build

misc
.DS_Store
.env.local
.env.development.local
.env.test.local
.env.production.local

```
npm-debug.log*
yarn-debug.log*
yarn-error.log*
</file>

<file path="package.json">
{
    "name": "frontend",
    "version": "0.1.0",
    "private": true,
    "dependencies": {
        "@ant-design/icons": "^6.1.0",
        "@testing-library/dom": "^10.4.1",
        "@testing-library/jest-dom": "^6.9.1",
        "@testing-library/react": "^16.3.2",
        "@testing-library/user-event": "^13.5.0",
        "antd": "^6.3.1",
        "dayjs": "^1.11.19",
        "jwt-decode": "^4.0.0",
        "lodash": "^4.17.23",
        "oidc-client-ts": "^3.5.0",
        "react": "^19.2.4",
        "react-dom": "^19.2.4",
        "react-router-dom": "^7.13.1",
        "react-scripts": "5.0.1",
        "web-vitals": "^2.1.4"
    },
    "scripts": {
        "start": "react-scripts start",
        "build": "react-scripts build",
        "test": "react-scripts test",
        "eject": "react-scripts eject"
    },
    "eslintConfig": {
        "extends": [
            "react-app",
            "react-app/jest"
        ]
    },
    "browserslist": {
        "production": [
            ">0.2%",
            "not dead",
            "not op_mini all"
        ],
        "development": [
            "last 1 chrome version",
            "last 1 firefox version",
            "last 1 safari version"
        ]
    }
}
</file>
```

<file path="public/index.html">
<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="utf-8" />
    <link rel="icon" href="%PUBLIC_URL%/favicon.ico" />
    <meta name="viewport" content="width=device-width, initial-scale=1" />
    <meta name="theme-color" content="#000000" />
    <meta
      name="description"
      content="Web site created using create-react-app"
    />
    <link rel="apple-touch-icon" href="%PUBLIC_URL%/logo192.png" />
    <!--
      manifest.json provides metadata used when your web app is installed on a
      user's mobile device or desktop. See
https://developers.google.com/web/fundamentals/web-app-manifest/
    -->
    <link rel="manifest" href="%PUBLIC_URL%/manifest.json" />
    <!--
      Notice the use of %PUBLIC_URL% in the tags above.
      It will be replaced with the URL of the `public` folder during the build.
      Only files inside the `public` folder can be referenced from the HTML.

      Unlike "/favicon.ico" or "favicon.ico", "%PUBLIC_URL%/favicon.ico" will
      work correctly both with client-side routing and a non-root public URL.
      Learn how to configure a non-root public URL by running `npm run build`.
    -->
    <title>React App</title>
  </head>
  <body>
    <noscript>You need to enable JavaScript to run this app.</noscript>

    <!--
      This HTML file is a template.
      If you open it directly in the browser, you will see an empty page.

      You can add webfonts, meta tags, or analytics to this file.
      The build step will place the bundled scripts into the <body> tag.

      To begin the development, run `npm start` or `yarn start`.
      To create a production bundle, use `npm run build` or `yarn build`.
    -->
  </body>
</html>
</file>

<file path="public/manifest.json">
{
  "short_name": "React App",
  "name": "Create React App Sample",
  "icons": [
    {
      "src": "favicon.ico",

```
      "sizes": "64x64 32x32 24x24 16x16",
      "type": "image/x-icon"
    },
    {
      "src": "logo192.png",
      "type": "image/png",
      "sizes": "192x192"
    },
    {
      "src": "logo512.png",
      "type": "image/png",
      "sizes": "512x512"
    }
  ],
  "start_url": ".",
  "display": "standalone",
  "theme_color": "#000000",
  "background_color": "#ffffff"
}
</file>

<file path="public/robots.txt">
# https://www.robotstxt.org/robotstxt.html
User-agent: *
Disallow:
</file>

<file path="README.md">
# Getting Started with Create React App

This project was bootstrapped with [Create React
App](https://github.com/facebook/create-react-app).

## Available Scripts

In the project directory, you can run:

### `npm start`

Runs the app in the development mode.\
Open [http://localhost:3000](http://localhost:3000) to view it in your browser.

The page will reload when you make changes.\
You may also see any lint errors in the console.

### `npm test`

Launches the test runner in the interactive watch mode.\
See the section about [running
tests](https://facebook.github.io/create-react-app/docs/running-tests) for more
information.

### `npm run build`
```

Builds the app for production to the `build` folder.\
It correctly bundles React in production mode and optimizes the build for the best performance.

The build is minified and the filenames include the hashes.\
Your app is ready to be deployed!

See the section about [deployment](https://facebook.github.io/create-react-app/docs/deployment) for more information.

`npm run eject`

Note: this is a one-way operation. Once you `eject`, you can't go back!

If you aren't satisfied with the build tool and configuration choices, you can `eject` at any time. This command will remove the single build dependency from your project.

Instead, it will copy all the configuration files and the transitive dependencies (webpack, Babel, ESLint, etc) right into your project so you have full control over them. All of the commands except `eject` will still work, but they will point to the copied scripts so you can tweak them. At this point you're on your own.

You don't have to ever use `eject`. The curated feature set is suitable for small and middle deployments, and you shouldn't feel obligated to use this feature. However we understand that this tool wouldn't be useful if you couldn't customize it when you are ready for it.

Learn More

You can learn more in the [Create React App documentation](https://facebook.github.io/create-react-app/docs/getting-started).

To learn React, check out the [React documentation](https://reactjs.org/).

Code Splitting

This section has moved here: https://facebook.github.io/create-react-app/docs/code-splitting

Analyzing the Bundle Size

This section has moved here: https://facebook.github.io/create-react-app/docs/analyzing-the-bundle-size

Making a Progressive Web App

This section has moved here: [https://facebook.github.io/create-react-app/docs/making-a-progressive-web-app](

https://facebook.github.io/create-react-app/docs/making-a-progressive-web-app)

Advanced Configuration

This section has moved here:
https://facebook.github.io/create-react-app/docs/advanced-configuration

Deployment

This section has moved here:
https://facebook.github.io/create-react-app/docs/deployment

`npm run build` fails to minify

This section has moved here:
https://facebook.github.io/create-react-app/docs/troubleshooting#npm-run-build-fails-to-minify
</file>

<file path="src/App.js">
```
import React from 'react';
import { BrowserRouter, Routes, Route } from 'react-router-dom';
import { AuthProvider, useAuth } from './context/authContext';
import { PrivateRoute } from './components/privateRoute/PrivateRoute';
import { PublicRoute } from './components/publicRoute/PublicRoute';
import { AppLayout } from './components/layout/AppLayout';
import { HomePage } from './pages/home/HomePage';
import { LoginPage } from './pages/login/LoginPage';
import { ProfilePage } from './pages/profile/ProfilePage';
import { NotFoundPage } from './pages/notFound/NotFoundPage';
import { Spin } from 'antd'; // для индикатора загрузки
import { ForbiddenPage } from './pages/forbidden/ForbiddenPage';
import { AccountsPage } from './pages/accounts/AccountsPage';
import { UsersPage } from './pages/users/UsersPage';
import { TariffsPage } from './pages/tariffs/TariffsPage';
import { CreditsPage } from './pages/credits/CreditsPage';
import Callback from './pages/login/Callback';

function AppRoutes() {
  const { loading } = useAuth();
  if (loading) {
    return (
      <div style={{ display: 'flex', justifyContent: 'center', marginTop: 100
}}>
        <Spin size="large" />
      </div>
    );
  }
  return (
    <Routes>
      <Route
```
</file>

```
  path="/"
  element={
    <PrivateRoute>
      <HomePage />
    </PrivateRoute>
  }
/>
<Route
  path="/login"
  element={
    <PublicRoute>
      <LoginPage />
    </PublicRoute>
  }
/>
<Route
  path="/profile"
  element={
    <PrivateRoute>
      <ProfilePage />
    </PrivateRoute>
  }
/>
<Route
  path="/accounts"
  element={
    <PrivateRoute>
      <AccountsPage />
    </PrivateRoute>
  }
/>
<Route
  path="/users"
  element={
    <PrivateRoute>
      <UsersPage />
    </PrivateRoute>
  }
/>
<Route
  path="/tariffs"
  element={
    <PrivateRoute>
      <TariffsPage />
    </PrivateRoute>
  }
/>
<Route
  path="/credits"
  element={
    <PrivateRoute>
      <CreditsPage />
    </PrivateRoute>
  }
```

```
      />
      <Route
        path="/signin-oidc"
        element={
            <Callback />
        }
      />
      <Route path="/forbidden" element={<ForbiddenPage/>} />
      <Route path="*" element={<NotFoundPage />} />
    </Routes>
  );
}

function App() {
  return (
    <AuthProvider>
      <BrowserRouter>
        <AppLayout>
          <AppRoutes />
        </AppLayout>
      </BrowserRouter>
    </AuthProvider>
  );
}

export default App;
</file>

<file path="src/components/layout/AppLayout.jsx">
import React from 'react';
import { Layout, Menu } from 'antd';
import { Link, useLocation } from 'react-router-dom';
import { useAuth } from '../../context/authContext';
import { AccountBookOutlined, ContactsOutlined, CreditCardOutlined,
ExceptionOutlined, HomeOutlined, ProfileOutlined, ReconciliationOutlined } from
'@ant-design/icons';

const { Header, Content, Footer } = Layout;

export const AppLayout = ({ children }) => {
  const location = useLocation();
  const { user, logout } = useAuth();

  // Если страница логина – показываем только содержимое (без шапки и футера)
  if (location.pathname === '/login') {
    return <>{children}</>;
  }

  const menuItems = [
    { key: '/', icon: <HomeOutlined/>, label: <Link to="/">Главная</Link> },
    { key: '/users', icon: <ContactsOutlined/>, label: <Link
to="/users">Пользователи</Link> },
    { key: '/accounts', icon: <AccountBookOutlined/>, label: <Link
to="/accounts">Счета</Link> },
```

```
    { key: '/tariffs', icon: <CreditCardOutlined />, label: <Link
to="/tariffs">Тарифы</Link> },
    { key: '/credits', icon: <ReconciliationOutlined />, label: <Link
to="/credits">Кредиты</Link> },
    user ? { key: '/profile', icon: <ProfileOutlined/>, label: <Link
to="/profile">Профиль</Link> } : null,
    user
      ? { key: 'logout', icon: <ExceptionOutlined/>, label: <span
onClick={logout}>Выйти</span> }
      : { key: '/login', label: <Link to="/login">Вход</Link> },
  ].filter(Boolean);

  return (
    <Layout className="layout" style={{ minHeight: '100vh' }}>
      <Header>
        <div className="logo" style={{ float: 'left', color: '#fff',
marginRight: 20 }}>
          Банк
        </div>
        <Menu
          theme="dark"
          mode="horizontal"
          selectedKeys={[location.pathname]}
          items={menuItems}
        />
      </Header>
      <Content style={{ padding: '0 50px', marginTop: 20 }}>
        <div className="site-layout-content" style={{ background: '#fff',
padding: 24, minHeight: 280 }}>
          {children}
        </div>
      </Content>
      <Footer style={{ textAlign: 'center' }}>Банк</Footer>
    </Layout>
  );
};
</file>

<file path="src/components/privateRoute/PrivateRoute.jsx">
import React from 'react';
import { Navigate } from 'react-router-dom';
import { getAccessToken, getRoleFromToken, isRoleAllowed } from
'../../services/api';
import { useAuth } from '../../context/authContext';

export const PrivateRoute = ({ children }) => {
    const { user, loading } = useAuth();

  if (!user) {
    return <Navigate to="/login" />;
  }

  const role = getRoleFromToken(user.access_token);
  if (!isRoleAllowed(role)) {
```

```
    return <Navigate to="/forbidden" />;
  }

  return children;
};
</file>

<file path="src/components/publicRoute/PublicRoute.jsx">
import React from 'react';
import { Navigate } from 'react-router-dom';
import { useAuth } from '../../context/authContext';

export const PublicRoute = ({ children }) => {
  const { user } = useAuth();
  // Если пользователь уже авторизован – перенаправляем на главную
  return user ? <Navigate to="/" /> : children;
};
</file>

<file path="src/config.js">
const config = {
  authority: 'http://localhost:2280', // URL IdentityServer
  client_id: 'web_client', // Client ID, зарегистрированный в IdentityServer
  redirect_uri: 'http://localhost:3000/signin-oidc', // Адрес после логина
  response_type: 'code',
  scope: 'openid profile account_api credit_api',
  post_logout_redirect_uri: 'http://localhost:3000/signout-callback-oidc',
};

export default config;
</file>

<file path="src/context/authContext.js">
import React, { createContext, useState, useEffect, useContext } from 'react';
import { getUser, login, logout } from '../services/authService';

const AuthContext = createContext();

export const AuthProvider = ({ children }) => {
  const [user, setUser] = useState(null);
  const [loading, setLoading] = useState(true);

  useEffect(() => {
    const initAuth = async () => {
      try {
        const currentUser = await getUser();
        setUser(currentUser);
      } catch (error) {
        console.error('Auth init error', error);
      } finally {
        setLoading(false);
      }
    };
    initAuth();
```

```
  }, []);

  const loginHandler = () => {
    login();
  };

  const logoutHandler = () => {
    logout();
  };

  const value = {
    user,
    loading,
    login: loginHandler,
    logout: logoutHandler,
  };

  return <AuthContext.Provider value={value}>{children}</AuthContext.Provider>;
};

export const useAuth = () => useContext(AuthContext);
</file>

<file path="src/index.js">
import React from 'react';
import ReactDOM from 'react-dom/client';
import App from './App';
import 'antd/dist/reset.css';

const root = ReactDOM.createRoot(document.getElementById('root'));
root.render(
  <React.StrictMode>
    <App />
  </React.StrictMode>
);
</file>

<file path="src/pages/accounts/AccountsPage.jsx">
import React, { useEffect, useState } from 'react';
import { Table, Tag, Button, message, Spin, Tooltip, Drawer, Space, Typography,
Popconfirm } from 'antd';
import { UserOutlined, DeleteOutlined, CloseOutlined, MoneyCollectOutlined,
MinusCircleOutlined, PlusCircleOutlined, BlockOutlined, EyeFilled, EyeOutlined,
BookOutlined } from '@ant-design/icons';
import { useAuth } from '../../context/authContext';
import { getAccounts, deleteAccount, getAccountTransactions } from
'../../services/core';
import { authApiRequest } from '../../services/api';
import dayjs from 'dayjs'; // если есть, иначе использовать new Date()
import { transactionColumns } from './values/transactionTable';
import { DepositWithdrawModal } from './ui/DepositWithdrawModal';

const { Text } = Typography;
```

```
export const AccountsPage = () => {
  const [accounts, setAccounts] = useState([]);
  const [loading, setLoading] = useState(false);
  const [usersMap, setUsersMap] = useState({});
  const [selectedAccount, setSelectedAccount] = useState(null);
  const [drawerVisible, setDrawerVisible] = useState(false);
  const [transactions, setTransactions] = useState([]);
  const [transactionsLoading, setTransactionsLoading] = useState(false);

  const [modalStatus, setModalStatus] = useState(null)

  const fetchAccounts = async () => {
    setLoading(true);
    try {
      const data = await getAccounts();
      setAccounts(data);

      // Загружаем данные владельцев
      const uniqueUserIds = [...new Set(data.map(acc =>
acc.userId).filter(Boolean))];
      if (uniqueUserIds.length > 0) {
        const usersData = await Promise.all(
          uniqueUserIds.map(async (id) => {
            try {
              const res = await authApiRequest(`/api/users/${id}`, { method:
'GET' });
              if (res.ok) return await res.json();
            } catch {
              return null;
            }
            return null;
          })
        );
        const map = {};
        usersData.forEach(u => {
          if (u && u.id) map[u.id] = u;
        });
        setUsersMap(map);
      }
    } catch (error) {
      message.error('Не удалось загрузить счета');
    } finally {
      setLoading(false);
    }
  };

  useEffect(() => {
    fetchAccounts();
  }, []);

  const handleDelete = async (id, e) => {
    e.stopPropagation(); // предотвращаем открытие drawer при клике на удаление
    try {
      await deleteAccount(id);
```

```
      message.success('Счёт удалён');
    } catch (e){
      message.error('Ошибка при удалении счёта: ' + e.message);
    }
      fetchAccounts();
  };

  const handleRowClick = (record) => {
    setSelectedAccount(record);
    setDrawerVisible(true);
    loadTransactions(record.id);
  };

  const loadTransactions = async (accountId) => {
    setTransactionsLoading(true);
    try {
      const data = await getAccountTransactions(accountId); // нужно добавить в
core.js
      setTransactions(data);
    } catch (error) {
      message.error('Не удалось загрузить транзакции');
      setTransactions([]);
    } finally {
      setTransactionsLoading(false);
    }
  };

  const closeDrawer = () => {
    setDrawerVisible(false);
    setSelectedAccount(null);
    setTransactions([]);
  };

  const invalidate = () => {
    fetchAccounts();
    if(selectedAccount?.id)
        loadTransactions(selectedAccount?.id)
  }

  const columns = [
    {
      title: 'ID счёта',
      dataIndex: 'id',
      key: 'id',
      render: (id) => <code>{id.substring(0, 8)}...</code>,
    },
    {
      title: 'Баланс',
      dataIndex: 'balance',
      key: 'balance',
      render: (balance) => <Tag color="green">{balance} ₽</Tag>,
    },
    {
      title: 'Статус',
```

```
    key: 'status',
    render: (_, record) => {
      if (!record.closedAt && !record.isDeleted) {
        return <Tag color="green">Активен</Tag>;
      } else if(record.closedAt) {
        const closedDate = dayjs(record.closedAt).format('DD.MM.YYYY HH:mm');
        return <Tag color="red">Закрыт {closedDate}</Tag>;
      }else if(record.isDeleted) {
        return <Tag color="red">Пользователь заблокирован</Tag>;
      }
    },
  },
  {
    title: 'Владелец',
    key: 'owner',
    render: (_, record) => {
      const owner = usersMap[record.userId];
      if (!owner) return '—';
      return (
        <Tooltip title={owner.email}>
          <UserOutlined /> {owner.credentials || owner.email}
        </Tooltip>
      );
    },
  },
  {
    title: 'Действия',
    key: 'actions',
    render: (_, record) => (
      <Space>
          <Button
              icon={<BookOutlined />}
              type='default'
              size="small"
              onClick={() => handleRowClick(record)}
          >История операций</Button>
          {!record.closedAt && !record.isDeleted && <Popconfirm
          title="Подтвердите действие"
          description="Вы уверены что хотите закрыть этот счёт?"
          okText="Закрыть"
          cancelText="Отмена"
          onConfirm={(e) => handleDelete(record.id, e)}>
              <Button
                  icon={<DeleteOutlined />}
                  danger
                  size="small"
              >Закрыть</Button>
          </Popconfirm> }
      </Space>
    ),
  },
];
```

```
  return (
    <div>
      <h2>Счета клиентов</h2>
      {loading ? (
        <Spin size="large" style={{ display: 'block', margin: '50px auto' }} />
      ) : (
        <Table
          dataSource={accounts}
          columns={columns}
          rowKey="id"
          pagination={{ pageSize: 10 }}
        />
      )}

      <Drawer
        title={`Транзакции счета ${selectedAccount?.id}`}
        placement="right"
        width={800}
        onClose={closeDrawer}
        open={drawerVisible}
        extra={
          <Space>
            <Button type='primary' onClick={() => setModalStatus({open: true,
type: 'deposit'})} icon={<PlusCircleOutlined/>}>Пополнить</Button>
            <Button type='default' onClick={() => setModalStatus({open: true,
type: 'withdraw'})} icon={<MinusCircleOutlined/>}>Снять</Button>
          </Space>
        }
      >
        {transactionsLoading ? (
          <Spin />
        ) : (
            <Table
                dataSource={transactions}
                columns={transactionColumns(transactions, selectedAccount)}
                rowKey="id"
                pagination={{ pageSize: 20 }}
                locale={{ emptyText: 'Нет транзакций' }}
            />
        )}
      </Drawer>

      <DepositWithdrawModal
        accountId={selectedAccount?.id}
        onInvalidate={invalidate}
        open={modalStatus?.open ?? false}
        type={modalStatus?.type ?? 'withdraw'}
        onClose={() => setModalStatus(null)}
      />
    </div>
  );
};
</file>
```

<file path="src/pages/accounts/ui/DepositWithdrawModal.jsx">
```
import { Modal, Form, Input, InputNumber, Button, message } from 'antd';
import { useSubmit } from 'react-router-dom';
import { useEffect } from 'react';
import { deposit, withdraw } from '../../../services/core';

export const DepositWithdrawModal = ({ open, onClose, onInvalidate, type,
accountId }) => {
  const [form] = Form.useForm();

  // Reset form when modal opens/closes or type changes
  useEffect(() => {
    if (open) {
      form.resetFields();
    }
  }, [open, form]);

  const handleFinish = async (values) => {
    const req = type == "deposit" ? deposit : withdraw;

    try{
        var json = await req(accountId, values);
    } catch (e) {
        message.error({ content: e.message })
    }
    onInvalidate();
    onClose();
  };

  const title = type === 'deposit' ? 'Пополнение счета' : 'Снятие со счета';
  const okText = type === 'deposit' ? 'Пополнить' : 'Снять';

  return (
    <Modal
      open={open}
      title={title}
      onCancel={onClose}
      footer={null} // We'll use the form's own buttons
      destroyOnClose
    >
      <Form
        form={form}
        layout="vertical"
        onFinish={handleFinish}
        initialValues={{ amount: undefined, description: '' }}
      >
        <Form.Item
          name="amount"
          label="Сумма"
          rules={[
            { required: true, message: 'Введите сумму' },
            { type: 'number', min: 0.01, message: 'Сумма должна быть больше 0'
},
```
</file>

```
          ]}
        >
          <InputNumber
            style={{ width: '100%' }}
            placeholder="0.00"
            precision={2}
            step={0.01}
            min={0.01}
          />
        </Form.Item>

        <Form.Item
          name="description"
          label="Описание (необязательно)"
        >
          <Input.TextArea rows={3} placeholder="Введите описание транзакции" />
        </Form.Item>

        <Form.Item style={{ marginBottom: 0, textAlign: 'right' }}>
          <Button onClick={onClose} style={{ marginRight: 8 }}>
            Отмена
          </Button>
          <Button type="primary" htmlType="submit">
            {okText}
          </Button>
        </Form.Item>
      </Form>
    </Modal>
  );
};
</file>

<file path="src/pages/accounts/values/transactionTable.js">
import { Tag } from 'antd';
import dayjs from 'dayjs';


const typeMap = {
    Unclassified: "неклассифицировано",
    Deposit: "Пополнение ч/з банкомат",
    Withdrawal: "Снятие ч/з банкомат",
    Transfer: "Перевод",
    CreditPayment: "Выплата кредита",
    CreditIncoming: "Взятие кредита"
}

// Колонки для таблицы транзакций с фильтрами
export const transactionColumns = (data, selectedAccount) => {
    // Вычисляем уникальные значения для фильтров
const typeFilters = [...new Set(data.map(t => t.displayType))]
  .map(type => ({
    text: typeMap[type] || type,
    value: type
  }));
```

```
const statusFilters = [...new Set(data.map(t => t.status))]
  .map(status => ({
    text: status,
    value: status
  }));
return[
  {
    title: 'Дата',
    dataIndex: 'createdAt',
    key: 'createdAt',
    render: (date) => dayjs(date).format('DD.MM.YYYY HH:mm'),
    // Можно добавить фильтр по дате с помощью filterDropdown,
    // но для простоты здесь не реализовано
  },
  {
    title: 'Тип',
    dataIndex: 'displayType',
    key: 'type',
    render: (type) => typeMap[type] || typeMap.Unclassified,
    filters: typeFilters,       // массив объектов { text, value }
    onFilter: (value, record) => record.type === value,
  },
  {
    title: 'Сумма',
    dataIndex: 'amount',
    key: 'amount',
    render: (amount, record) => (
      <Tag color={record.targetId == selectedAccount.id ? 'green' : 'red'}>
        {record.targetId == selectedAccount.id ? '+' : '-'} {amount} ₽
      </Tag>
    ),
    // Для числовых фильтров можно использовать filterDropdown с RangePicker,
    // но здесь не реализовано для краткости
  },
  {
    title: 'Описание',
    dataIndex: 'description',
    key: 'description',
    // Можно добавить текстовый поиск через глобальный поиск таблицы,
    // либо через filterDropdown с Input.Search
  },
  {
    title: 'Статус',
    dataIndex: 'status',
    key: 'status',
    render: (status) => (
      <Tag color={{ Pending: 'yellow', Completed: 'green', Failed: 'red'
}[status]}>
        {status}
      </Tag>
    ),
    filters: statusFilters,
    onFilter: (value, record) => record.status === value,
```

```
  },
]};
</file>

<file path="src/pages/credits/CreditsPage.jsx">
// pages/credits/CreditsPage.jsx
import React, { useEffect, useState } from 'react';
import {
  Table,
  Tag,
  Button,
  message,
  Spin,
  Drawer,
  Space,
  Typography,
  Modal,
  Form,
  Input,
  InputNumber,
  Tooltip,
} from 'antd';
import {
  EyeOutlined,
  CheckOutlined,
  CloseOutlined,
  UserOutlined,
} from '@ant-design/icons';
import { useAuth } from '../../context/authContext';
import { getCredits, approveCredit, rejectCredit, getCreditById } from
'../../services/credit';
import { authApiRequest } from '../../services/api';
import dayjs from 'dayjs';

const { Text } = Typography;

const statusColors = {
  Pending: 'orange',
  Approved: 'green',
  Rejected: 'red',
  Closed: 'gray',
};

export const CreditsPage = () => {
  const [credits, setCredits] = useState([]);
  const [loading, setLoading] = useState(false);
  const [usersMap, setUsersMap] = useState({});

  const [selectedCredit, setSelectedCredit] = useState(null);
  const [drawerVisible, setDrawerVisible] = useState(false);
  const [detailsLoading, setDetailsLoading] = useState(false);

  const [approveModalVisible, setApproveModalVisible] = useState(false);
  const [rejectModalVisible, setRejectModalVisible] = useState(false);
```

```
  const [actionLoading, setActionLoading] = useState(false);

  const [approveForm] = Form.useForm();
  const [rejectForm] = Form.useForm();

  const fetchCredits = async () => {
    setLoading(true);
    try {
      const data = await getCredits();
      setCredits(data);

      // Загружаем данные владельцев
      const uniqueUserIds = [...new Set(data.map(c =>
c.userId).filter(Boolean))];
      if (uniqueUserIds.length > 0) {
        const usersData = await Promise.all(
          uniqueUserIds.map(async (id) => {
            try {
              const res = await authApiRequest(`/api/users/${id}`, { method:
'GET' });
              if (res.ok) return await res.json();
            } catch {
              return null;
            }
            return null;
          })
        );
        const map = {};
        usersData.forEach(u => {
          if (u && u.id) map[u.id] = u;
        });
        setUsersMap(map);
      }
    } catch (error) {
      message.error('Не удалось загрузить кредиты: ' + error.message);
    } finally {
      setLoading(false);
    }
  };

  useEffect(() => {
    fetchCredits();
  }, []);

  const handleRowClick = async (record) => {
    setSelectedCredit(record);
    setDrawerVisible(true);
    // Можно загрузить свежие данные, но пока используем то, что есть
    // Если нужно точнее, можно вызвать getCreditById(record.id)
  };

  const closeDrawer = () => {
    setDrawerVisible(false);
    setSelectedCredit(null);
```

```
};

const handleApprove = async (values) => {
  setActionLoading(true);
  try {
    await approveCredit(selectedCredit.id, values);
    message.success('Кредит одобрен');
    setApproveModalVisible(false);
    approveForm.resetFields();
    // Обновляем список и детали
    fetchCredits();
    if (selectedCredit) {
      const updated = await getCreditById(selectedCredit.id);
      setSelectedCredit(updated);
    }
  } catch (error) {
    message.error('Ошибка при одобрении: ' + error.message);
  } finally {
    setActionLoading(false);
  }
};

const handleReject = async (values) => {
  setActionLoading(true);
  try {
    await rejectCredit(selectedCredit.id, values);
    message.success('Кредит отклонён');
    setRejectModalVisible(false);
    rejectForm.resetFields();
    fetchCredits();
    if (selectedCredit) {
      const updated = await getCreditById(selectedCredit.id);
      setSelectedCredit(updated);
    }
  } catch (error) {
    message.error('Ошибка при отклонении: ' + error.message);
  } finally {
    setActionLoading(false);
  }
};

const columns = [
  {
    title: 'ID кредита',
    dataIndex: 'id',
    key: 'id',
    render: (id) => <code>{id.substring(0, 8)}...</code>,
  },
  {
    title: 'Выдан',
    key: 'user',
    render: (_, record) => {
      const owner = usersMap[record.userId];
      if (!owner) return <Text type="secondary">—</Text>;
```

```
    return (
      <Tooltip title={owner.email}>
        <Space>
            <UserOutlined />
            {owner.credentials || owner.email}
        </Space>
      </Tooltip>
    );
  },
},
{
  title: 'Сумма',
  dataIndex: 'amount',
  key: 'amount',
  render: (val) => `${val.toLocaleString()} ₽`,
},
{
  title: 'Остаток долга',
  dataIndex: 'remainingDebt',
  key: 'remainingDebt',
  render: (val) => `${val.toLocaleString()} ₽`,
},
{
  title: 'Срок (дней)',
  dataIndex: 'termDays',
  key: 'termDays',
},
{
  title: 'Статус',
  dataIndex: 'status',
  key: 'status',
  render: (status) => <Tag color={statusColors[status]}>{status}</Tag>,
},
{
  title: 'Дата создания',
  dataIndex: 'createdAt',
  key: 'createdAt',
  render: (date) => dayjs(date).format('DD.MM.YYYY HH:mm'),
},
{
  title: 'Действия',
  key: 'actions',
  render: (_, record) => (
    <Button
      icon={<EyeOutlined />}
      size="small"
      onClick={(e) => {
        e.stopPropagation();
        handleRowClick(record);
      }}
    >
      Просмотр
    </Button>
  ),
```

```
    },
  ];

  return (
    <div>
      <h2>Управление кредитами</h2>
      {loading ? (
        <Spin size="large" style={{ display: 'block', margin: '50px auto' }} />
      ) : (
        <Table
          dataSource={credits}
          columns={columns}
          rowKey="id"
          pagination={{ pageSize: 10 }}
          onRow={(record) => ({
            onClick: () => handleRowClick(record),
            style: { cursor: 'pointer' },
          })}
        />
      )}

      <Drawer
        title={`Детали кредита ${selectedCredit?.id.substring(0, 8)}...`}
        placement="right"
        width={600}
        onClose={closeDrawer}
        open={drawerVisible}
      >
        {selectedCredit ? (
          <Space direction="vertical" size="middle" style={{ width: '100%' }}>
            <div>
              <Text strong>ID:</Text> <Text>{selectedCredit.id}</Text>
            </div>
            <div>
              <Text strong>Владелец:</Text>{' '}
              <Text>
                {usersMap[selectedCredit.userId]?.email ||
selectedCredit.userId}
              </Text>
            </div>
            <div>
              <Text strong>Сумма:</Text> <Text>{selectedCredit.amount} ₽</Text>
            </div>
            <div>
              <Text strong>Остаток долга:</Text>
<Text>{selectedCredit.remainingDebt} ₽</Text>
            </div>
            <div>
              <Text strong>Срок (дней):</Text>
<Text>{selectedCredit.termDays}</Text>
            </div>
            <div>
              <Text strong>Статус:</Text>{' '}
              <Tag
```

```
color={statusColors[selectedCredit.status]}>{selectedCredit.status}</Tag>
            </div>
            <div>
              <Text strong>Дата создания:</Text>{' '}
              <Text>{dayjs(selectedCredit.createdAt).format('DD.MM.YYYY
HH:mm')}</Text>
            </div>
            {selectedCredit.approvedAmount && (
              <div>
                <Text strong>Одобренная сумма:</Text>
<Text>{selectedCredit.approvedAmount} ₽</Text>
              </div>
            )}
            {selectedCredit.approvedBy && (
              <div>
                <Text strong>Кем одобрен:</Text>
<Text>{selectedCredit.approvedBy}</Text>
              </div>
            )}
            {selectedCredit.rejectionReason && (
              <div>
                <Text strong>Причина отказа:</Text>
<Text>{selectedCredit.rejectionReason}</Text>
              </div>
            )}

            {selectedCredit.status === 'Pending' && (
              <Space style={{ marginTop: 20 }}>
                <Button
                  type="primary"
                  icon={<CheckOutlined />}
                  onClick={() => setApproveModalVisible(true)}
                >
                  Одобрить
                </Button>
                <Button
                  danger
                  icon={<CloseOutlined />}
                  onClick={() => setRejectModalVisible(true)}
                >
                  Отказать
                </Button>
              </Space>
            )}
          </Space>
        ) : (
          <Text>Кредит не выбран</Text>
        )}
      </Drawer>

      {/* Модальное окно одобрения */}
      <Modal
        title="Одобрение кредита"
        open={approveModalVisible}
```

```
        onCancel={() => setApproveModalVisible(false)}
        footer={null}
        destroyOnClose
      >
        <Form form={approveForm} layout="vertical" onFinish={handleApprove}>
          <Form.Item name="approvedAmount" label="Одобренная сумма (оставьте
пустым для суммы заявки)">
            <InputNumber
              style={{ width: '100%' }}
              placeholder="Сумма"
              min={0.01}
              step={0.01}
              precision={2}
            />
          </Form.Item>
          <Form.Item name="comment" label="Комментарий">
            <Input.TextArea rows={3} placeholder="Комментарий (необязательно)"
/>
          </Form.Item>
          <Form.Item style={{ textAlign: 'right' }}>
            <Space>
              <Button onClick={() =>
setApproveModalVisible(false)}>Отмена</Button>
              <Button type="primary" htmlType="submit" loading={actionLoading}>
                Одобрить
              </Button>
            </Space>
          </Form.Item>
        </Form>
      </Modal>

      {/* Модальное окно отказа */}
      <Modal
        title="Отказ в кредите"
        open={rejectModalVisible}
        onCancel={() => setRejectModalVisible(false)}
        footer={null}
        destroyOnClose
      >
        <Form form={rejectForm} layout="vertical" onFinish={handleReject}>
          <Form.Item name="reason" label="Причина отказа" rules={[{ required:
true, message: 'Введите причину' }]}>
            <Input.TextArea rows={3} placeholder="Причина отказа" />
          </Form.Item>
          <Form.Item style={{ textAlign: 'right' }}>
            <Space>
              <Button onClick={() =>
setRejectModalVisible(false)}>Отмена</Button>
              <Button type="primary" danger htmlType="submit"
loading={actionLoading}>
                Отказать
              </Button>
            </Space>
          </Form.Item>
```

```
        </Form>
      </Modal>
    </div>
  );
};
</file>

<file path="src/pages/forbidden/ForbiddenPage.jsx">
import React from 'react';
import { Result, Button } from 'antd';
import { Link } from 'react-router-dom';
import { useAuth } from '../../context/authContext';

export const ForbiddenPage = () => {
  const { logout } = useAuth();
  return (
    <Result
      status="403"
      title="403"
      subTitle="Извините, у вас нет доступа к этой странице."
      extra={[
        <Button type="primary" key="home">
          <Link to="/">На главную</Link>
        </Button>,
        <Button key="logout" onClick={logout}>
          Выйти
        </Button>,
      ]}
    />
  );
};
</file>

<file path="src/pages/home/HomePage.jsx">
import React from 'react';
import { Typography, Card, Row, Col, Space } from 'antd';
import { UserOutlined, AccountBookOutlined, CreditCardOutlined,
ReconciliationOutlined } from '@ant-design/icons';
import { Link } from 'react-router-dom';
import { useAuth } from '../../context/authContext';

const { Title, Paragraph } = Typography;

export const HomePage = () => {
  const { user } = useAuth();

  const services = [
    {
      title: 'Пользователи',
      icon: <UserOutlined style={{ fontSize: 48, color: '#1890ff' }} />,
      link: '/users',
      description: 'Управление пользователями, блокировка, назначение ролей',
    },
    {
```

```
      title: 'Счета',
      icon: <AccountBookOutlined style={{ fontSize: 48, color: '#52c41a' }} />,
      link: '/accounts',
      description: 'Просмотр счетов, транзакции, пополнение и снятие',
    },
    {
      title: 'Тарифы',
      icon: <CreditCardOutlined style={{ fontSize: 48, color: '#faad14' }} />,
      link: '/tariffs',
      description: 'Управление тарифами по кредитам',
    },
    {
      title: 'Кредиты',
      icon: <ReconciliationOutlined style={{ fontSize: 48, color: '#722ed1' }}
/>,
      link: '/credits',
      description: 'Просмотр кредитных заявок, одобрение и отказ',
    },
  ];

  return (
    <div>
      <Title level={2}>Добро пожаловать, {user?.credentials ||
user?.email}!</Title>
      <Paragraph type="secondary">
        Это панель управления банковской системой. Выберите раздел для работы.
      </Paragraph>

      <Row gutter={[24, 24]} style={{ marginTop: 32 }}>
        {services.map((service) => (
          <Col xs={24} sm={12} md={8} lg={6} key={service.link}>
            <Link to={service.link}>
              <Card
                hoverable
                style={{ textAlign: 'center', height: '100%' }}
                cover={<div style={{ padding: '20px 0' }}>{service.icon}</div>}
              >
                <Card.Meta
                  title={service.title}
                  description={service.description}
                />
              </Card>
            </Link>
          </Col>
        ))}
      </Row>
    </div>
  );
};
</file>

<file path="src/pages/login/Callback.jsx">
import React, { useEffect } from 'react';
import { useNavigate } from 'react-router-dom';
```

```
import { loginCallback } from '../../services/authService';

const Callback = () => {
  const navigate = useNavigate();

  useEffect(() => {
    const handleCallback = async () => {
      try {
        await loginCallback();
        navigate('/');
      } catch (error) {
        console.error('Callback error', error);
        navigate('/login');
      }
    };
    handleCallback();
  }, [navigate]);

  return <div>Loading...</div>;
};

export default Callback;
</file>

<file path="src/pages/login/LoginPage.jsx">
import React from 'react';
import { useAuth } from '../../context/authContext';

export const LoginPage = () => {
  const { login } = useAuth();
  return (
    <div>
      <h1>Login</h1>
      <button onClick={login}>Sign in with IdentityServer</button>
    </div>
  );
};
</file>

<file path="src/pages/notFound/NotFoundPage.jsx">
import React from 'react';
import { Result, Button } from 'antd';
import { Link } from 'react-router-dom';

export const NotFoundPage = () => {
  return (
    <Result
      status="404"
      title="404"
      subTitle="Извините, страница не найдена."
      extra={<Button type="primary"><Link to="/">Вернуться на
главную</Link></Button>}
    />
  );
```

```
};
</file>

<file path="src/pages/profile/ProfilePage.jsx">
import React, { useState } from 'react';
import { Typography, Card, Button, Form, Input, message, Tabs } from 'antd';
import { useAuth } from '../../context/authContext';

const { Title, Text } = Typography;
const { TabPane } = Tabs;

export const ProfilePage = () => {
  const { user, logout, updateProfile, changePassword } = useAuth();
  const [profileForm] = Form.useForm();
  const [passwordForm] = Form.useForm();
  const [updating, setUpdating] = useState(false);
  const [changing, setChanging] = useState(false);

  const handleUpdateProfile = async (values) => {
    setUpdating(true);
    const result = await updateProfile(values);
    setUpdating(false);
    if (result.success) {
      message.success('Профиль обновлён');
    } else {
      message.error(result.error);
    }
  };

  const handleChangePassword = async (values) => {
    setChanging(true);
    const result = await changePassword(values.oldPassword, values.newPassword);
    setChanging(false);
    if (result.success) {
      message.success('Пароль изменён');
      passwordForm.resetFields();
    } else {
      message.error(result.error);
    }
  };

  return (
    <Card style={{ maxWidth: 600, margin: '0 auto' }}>
      <Title level={2}>Профиль</Title>
      <Tabs defaultActiveKey="1">
        <TabPane tab="Основное" key="1">
          <div style={{ marginBottom: 20 }}>
            <Text strong>Email: </Text> <Text>{user?.email}</Text><br />
            <Text strong>Имя: </Text> <Text>{user?.credentials}</Text>
          </div>
          <Form
            form={profileForm}
            layout="vertical"
            onFinish={handleUpdateProfile}
```

```
            initialValues={{ email: user?.email, credentials: user?.credentials
}}
          >
            <Form.Item
              name="email"
              label="Email"
              rules={[{ required: true, type: 'email', message: 'Введите
корректный email' }]}
            >
              <Input />
            </Form.Item>
            <Form.Item
              name="credentials"
              label="Имя"
              rules={[{ required: true, message: 'Введите имя' }]}
            >
              <Input />
            </Form.Item>
            <Form.Item>
              <Button type="primary" htmlType="submit" loading={updating}>
                Обновить профиль
              </Button>
            </Form.Item>
          </Form>
        </TabPane>
        <TabPane tab="Смена пароля" key="2">
          <Form
            form={passwordForm}
            layout="vertical"
            onFinish={handleChangePassword}
          >
            <Form.Item
              name="oldPassword"
              label="Старый пароль"
              rules={[{ required: true, message: 'Введите старый пароль' }]}
            >
              <Input.Password />
            </Form.Item>
            <Form.Item
              name="newPassword"
              label="Новый пароль"
              rules={[{ required: true, message: 'Введите новый пароль' }]}
            >
              <Input.Password />
            </Form.Item>
            <Form.Item>
              <Button type="primary" htmlType="submit" loading={changing}>
                Сменить пароль
              </Button>
            </Form.Item>
          </Form>
        </TabPane>
      </Tabs>
      <div style={{ marginTop: 20 }}>
```

```
        <Button type="primary" danger onClick={logout}>
          Выйти
        </Button>
      </div>
    </Card>
  );
};
</file>

<file path="src/pages/tariffs/TariffsPage.jsx">
// pages/tariffs/TariffsPage.jsx
import React, { useEffect, useState } from 'react';
import {
  Table,
  Button,
  Space,
  Popconfirm,
  message,
  Modal,
  Form,
  Input,
  InputNumber,
  Typography,
  Spin,
} from 'antd';
import { PlusOutlined, EditOutlined, DeleteOutlined } from '@ant-design/icons';
import { getTariffs, createTariff, updateTariff, deleteTariff } from
'../../services/credit';

const { Title } = Typography;

export const TariffsPage = () => {
  const [tariffs, setTariffs] = useState([]);
  const [loading, setLoading] = useState(false);
  const [modalVisible, setModalVisible] = useState(false);
  const [editingTariff, setEditingTariff] = useState(null); // null – создание,
объект – редактирование
  const [form] = Form.useForm();
  const [submitting, setSubmitting] = useState(false);

  // Загрузка списка тарифов
  const fetchTariffs = async () => {
    setLoading(true);
    try {
      const data = await getTariffs();
      setTariffs(data);
    } catch (error) {
      message.error(error.message);
    } finally {
      setLoading(false);
    }
  };

  useEffect(() => {
```

```
  fetchTariffs();
}, []);

// Открыть модалку для создания
const handleCreate = () => {
  setEditingTariff(null);
  form.resetFields();
  setModalVisible(true);
};

// Открыть модалку для редактирования
const handleEdit = (record) => {
  setEditingTariff(record);
  form.setFieldsValue({
    name: record.name,
    interestRate: record.interestRate,
    maxAmount: record.maxAmount,
    maxTermDays: record.maxTermDays,
  });
  setModalVisible(true);
};

// Удаление тарифа
const handleDelete = async (id) => {
  try {
    await deleteTariff(id);
    message.success('Тариф удалён');
    fetchTariffs(); // обновляем список
  } catch (error) {
    message.error(error.message);
  }
};

// Сохранение (создание или обновление)
const handleSave = async (values) => {
  setSubmitting(true);
  try {
    if (editingTariff) {
      await updateTariff(editingTariff.id, values);
      message.success('Тариф обновлён');
    } else {
      await createTariff(values);
      message.success('Тариф создан');
    }
    setModalVisible(false);
    fetchTariffs();
  } catch (error) {
    message.error(error.message);
  } finally {
    setSubmitting(false);
  }
};

// Колонки таблицы
```

```
const columns = [
  {
    title: 'Название',
    dataIndex: 'name',
    key: 'name',
  },
  {
    title: 'Ставка %',
    dataIndex: 'interestRate',
    key: 'interestRate',
    render: (value) => `${value} %`,
  },
  {
    title: 'Макс. сумма',
    dataIndex: 'maxAmount',
    key: 'maxAmount',
    render: (value) => `${value.toLocaleString()} ₽`,
  },
  {
    title: 'Срок (дней)',
    dataIndex: 'maxTermDays',
    key: 'maxTermDays',
    render: (value) => `${value} дн.`,
  },
  {
    title: 'Действия',
    key: 'actions',
    render: (_, record) => (
      <Space>
        <Button
          icon={<EditOutlined />}
          size="small"
          onClick={() => handleEdit(record)}
        >
          Редактировать
        </Button>
        <Popconfirm
          title="Удалить тариф?"
          description="Вы уверены, что хотите удалить этот тариф?"
          okText="Да"
          cancelText="Нет"
          onConfirm={() => handleDelete(record.id)}
        >
          <Button icon={<DeleteOutlined />} size="small" danger>
            Удалить
          </Button>
        </Popconfirm>
      </Space>
    ),
  },
];

return (
  <div>
```

```
      <Title level={2}>Управление тарифами</Title>
      <Button
        type="primary"
        icon={<PlusOutlined />}
        onClick={handleCreate}
        style={{ marginBottom: 16 }}
      >
        Создать тариф
      </Button>

      {loading ? (
        <Spin size="large" style={{ display: 'block', margin: '50px auto' }} />
      ) : (
        <Table
          dataSource={tariffs}
          columns={columns}
          rowKey="id"
          pagination={{ pageSize: 10 }}
        />
      )}

      {/* Модальное окно создания/редактирования */}
      <Modal
        open={modalVisible}
        title={editingTariff ? 'Редактировать тариф' : 'Создать тариф'}
        onCancel={() => setModalVisible(false)}
        footer={null}
        destroyOnClose
      >
        <Form
          form={form}
          layout="vertical"
          onFinish={handleSave}
          initialValues={{
            interestRate: undefined,
            maxAmount: undefined,
            maxTermDays: undefined,
          }}
        >
          <Form.Item
            name="name"
            label="Название"
            rules={[{ required: true, message: 'Введите название тарифа' }]}
          >
            <Input />
          </Form.Item>

          <Form.Item
            name="interestRate"
            label="Процентная ставка (%)"
            rules={[
              { required: true, message: 'Введите ставку' },
              { type: 'number', min: 0.01, max: 100, message: 'Ставка от 0.01 до
100' },
```

```
            ]}
          >
            <InputNumber style={{ width: '100%' }} step={0.1} precision={2} />
          </Form.Item>

          <Form.Item
            name="maxAmount"
            label="Максимальная сумма"
            rules={[
              { required: true, message: 'Введите максимальную сумму' },
              { type: 'number', min: 0.01, message: 'Сумма должна быть больше 0'
},
            ]}
          >
            <InputNumber style={{ width: '100%' }} step={1000} precision={2} />
          </Form.Item>

          <Form.Item
            name="maxTermDays"
            label="Максимальный срок (дней)"
            rules={[
              { required: true, message: 'Введите срок в днях' },
              { type: 'number', min: 1, message: 'Срок должен быть не менее 1
дня' },
            ]}
          >
            <InputNumber style={{ width: '100%' }} step={1} precision={0} />
          </Form.Item>

          <Form.Item style={{ marginBottom: 0, textAlign: 'right' }}>
            <Space>
              <Button onClick={() => setModalVisible(false)}>Отмена</Button>
              <Button type="primary" htmlType="submit" loading={submitting}>
                {editingTariff ? 'Сохранить' : 'Создать'}
              </Button>
            </Space>
          </Form.Item>
        </Form>
      </Modal>
    </div>
  );
};
</file>

<file path="src/pages/users/UsersPage.jsx">
// pages/users/UsersPage.jsx
import React, { useEffect, useState, useCallback } from 'react';
import { Table, Input, Button, Drawer, Spin, message, Typography, Space,
Divider, Tag } from 'antd';
import { SearchOutlined, EyeOutlined, UnlockOutlined, LockOutlined } from
'@ant-design/icons';
import { useAuth } from '../../context/authContext';
import { searchUsers, getUserById } from '../../services/users';
import debounce from 'lodash/debounce';
```

```
import { addUserRole, removeUserRole, blockUser, unblockUser } from
'../../services/users';

const { Title, Text } = Typography;

export const UsersPage = () => {
  const [users, setUsers] = useState([]);
  const [loading, setLoading] = useState(false);
  const [searchQuery, setSearchQuery] = useState('');
  const [selectedUser, setSelectedUser] = useState(null);
  const [drawerVisible, setDrawerVisible] = useState(false);
  const [userDetailsLoading, setUserDetailsLoading] = useState(false);
  const [roleActionLoading, setRoleActionLoading] = useState(false);
const [blockActionLoading, setBlockActionLoading] = useState(false);

    const { user } = useAuth();

  // Загрузка пользователей с учётом поискового запроса
  const fetchUsers = async (query) => {
    setLoading(true);
    try {
      const data = await searchUsers(query);
      setUsers(data);
    } catch (error) {
      message.error(error.message || 'Ошибка загрузки пользователей');
    } finally {
      setLoading(false);
    }
  };

  // Первоначальная загрузка (пустой запрос — все пользователи)
  useEffect(() => {
    fetchUsers('');
  }, []);

  // Debounced поиск (ждём 500 мс после остановки ввода)
  const debouncedSearch = useCallback(
    debounce((query) => {
      fetchUsers(query);
    }, 500),
    []
  );

  const handleSearchChange = (e) => {
    const value = e.target.value;
    setSearchQuery(value);
    debouncedSearch(value);
  };

  // Открытие Drawer с загрузкой деталей пользователя
  const showUserDetails = async (user) => {
    setSelectedUser(user);
    setDrawerVisible(true);
    setUserDetailsLoading(true);
```

```
    try {
      // Можно повторно запросить свежие данные, но можно использовать уже
имеющиеся
      // Для простоты используем то, что есть в таблице
      // Если нужно точнее — раскомментируйте:
      // const detailed = await getUserById(user.id);
      // setSelectedUser(detailed);
    } catch (error) {
      message.error('Не удалось загрузить детали пользователя');
    } finally {
      setUserDetailsLoading(false);
    }
  };

  const closeDrawer = () => {
    setDrawerVisible(false);
    setSelectedUser(null);
  };

  const handleBlockUser = async (userId) => {
  setBlockActionLoading(true);
  try {
    await blockUser(userId);
    message.success('Пользователь заблокирован');
    const updated = await getUserById(userId);
    setSelectedUser(updated);
  } catch (error) {
    message.error(error.message || 'Ошибка блокировки');
  } finally {
    setBlockActionLoading(false);
  }
};

const handleUnblockUser = async (userId) => {
  setBlockActionLoading(true);
  try {
    await unblockUser(userId);
    message.success('Пользователь разблокирован');
    const updated = await getUserById(userId);
    setSelectedUser(updated);
  } catch (error) {
    message.error(error.message || 'Ошибка разблокировки');
  } finally {
    setBlockActionLoading(false);
  }
};

  const columns = [
    {
      title: 'ID',
      dataIndex: 'id',
      key: 'id',
      render: (id) => <code>{id.substring(0, 8)}...</code>,
    },
```

```
  {
    title: 'Email',
    dataIndex: 'email',
    key: 'email',
  },
  {
    title: 'Имя',
    dataIndex: 'credentials',
    key: 'credentials',
    render: (text) => text || '—',
  },
  {
    title: 'Действия',
    key: 'actions',
    render: (_, record) => (
      <Button
        icon={<EyeOutlined />}
        size="small"
        onClick={(e) => {
          e.stopPropagation(); // предотвращаем срабатывание клика по строке
          showUserDetails(record);
        }}
      >
        Просмотр
      </Button>
    ),
  },
];

  const handleAssignEmployee = async (userId) => {
  setRoleActionLoading(true);
  try {
      await addUserRole(userId, 'Employee');
      message.success('Роль Employee назначена');
  } catch (error) {
      message.error(error.message || 'Ошибка назначения роли');
  } finally {
      setRoleActionLoading(false);
      fetchUsers(searchQuery);
  }
  };

  const handleRemoveEmployee = async (userId) => {
  setRoleActionLoading(true);
  try {
      await removeUserRole(userId, 'Employee');
      message.success('Роль Employee снята');
  } catch (error) {
      message.error(error.message || 'Ошибка снятия роли');
  } finally {
      setRoleActionLoading(false);
      fetchUsers(searchQuery);
  }
  };
```

```
  return (
    <div>
      <Title level={2}>Пользователи</Title>

      {/* Поле поиска */}
      <Input
        placeholder="Поиск по имени"
        prefix={<SearchOutlined />}
        value={searchQuery}
        onChange={handleSearchChange}
        style={{ width: 300, marginBottom: 20 }}
        allowClear
      />

      {loading ? (
        <Spin size="large" style={{ display: 'block', margin: '50px auto' }} />
      ) : (
        <Table
          dataSource={users}
          columns={columns}
          rowKey="id"
          pagination={{ pageSize: 10 }}
          onRow={(record) => ({
            onClick: () => showUserDetails(record),
            style: { cursor: 'pointer' },
          })}
        />
      )}

      {/* Боковая панель с деталями пользователя */}
      <Drawer
        title="Информация о пользователе"
        placement="right"
        width={500}
        onClose={closeDrawer}
        open={drawerVisible}
      >
        {userDetailsLoading ? (
          <Spin />
        ) : selectedUser ? (
          <Space direction="vertical" size="middle" style={{ width: '100%' }}>
            <div>
              <Text strong>ID:</Text> <Text>{selectedUser.id}</Text>
            </div>
            <div>
              <Text strong>Email:</Text> <Text>{selectedUser.email}</Text>
            </div>
            <div>
              <Text strong>Имя:</Text> <Text>{selectedUser.credentials || '—'}
</Text>
            </div>

                  {/* Статус блокировки */}
```

```
<div>
  <Text strong>Статус:</Text>{' '}
  {selectedUser.isBlocked ? (
    <Tag color="red">Заблокирован</Tag>
  ) : (
    <Tag color="green">Активен</Tag>
  )}
</div>

{/* Роли */}
<div>
  <Text strong>Роли:</Text>{' '}
  {selectedUser.roles && selectedUser.roles.length > 0 ? (
    selectedUser.roles.map(role => (
      <Tag key={role} color="blue">{role}</Tag>
    ))
  ) : (
    <Text type="secondary">Нет ролей</Text>
  )}
</div>

<Divider />

{/* Управление ролью Employee */}
{user.id != selectedUser.id && <Space>
  {selectedUser.roles?.includes('Employee') ? (
    <Button
      danger
      onClick={() => handleRemoveEmployee(selectedUser.id)}
      loading={roleActionLoading}
    >
      Снять роль Employee
    </Button>
  ) : (
    <Button
      type="primary"
      onClick={() => handleAssignEmployee(selectedUser.id)}
      loading={roleActionLoading}
    >
      Назначить Employee
    </Button>
  )}
</Space>}

{/* Управление блокировкой (если доступно) */}
{user.id != selectedUser.id &&<Space style={{ marginTop: 16 }}>
  {selectedUser.isBlocked ? (
    <Button
      icon={<UnlockOutlined />}
      onClick={() => handleUnblockUser(selectedUser.id)}
      loading={blockActionLoading}
    >
      Разблокировать
    </Button>
```

```
        ) : (
          <Button
            icon={<LockOutlined />}
            danger
            onClick={() => handleBlockUser(selectedUser.id)}
            loading={blockActionLoading}
          >
            Заблокировать
          </Button>
        )}
      </Space> }
          </Space>
        ) : (
          <Text>Пользователь не выбран</Text>
        )}
      </Drawer>
    </div>
  );
};
</file>

<file path="src/services/api.js">
// services/api.js
import { jwtDecode } from 'jwt-decode';
import { getUser } from './authService';

// Функции для работы с токенами (общие)
const getAccessToken = () => localStorage.getItem('accessToken');
const getRefreshToken = () => localStorage.getItem('refreshToken');

const setTokens = (accessToken, refreshToken) => {
  localStorage.setItem('accessToken', accessToken);
  if (refreshToken) localStorage.setItem('refreshToken', refreshToken);
};

const removeTokens = () => {
  localStorage.removeItem('accessToken');
  localStorage.removeItem('refreshToken');
};

const getRoleFromToken = (token) => {
  try {
    const decoded = jwtDecode(token);
    return decoded.role; // предполагается поле "role"
  } catch {
    return null;
  }
};

const isRoleAllowed = (role) => role !== 'Customer';

// Фабрика для создания API-клиента с заданным baseURL
const createApiRequest = (baseURL) => {
  return async function apiRequest(endpoint, options = {}) {
```

```
    const url = `${baseURL}${endpoint}`;
    const user = await getUser();
    const accessToken = user.access_token;

    const headers = {
      'Content-Type': 'application/json',
      ...options.headers,
    };

    if (accessToken) {
      headers.Authorization = `Bearer ${accessToken}`;
    }

    let response = await fetch(url, { ...options, headers });

    /*// Попытка обновить токен при 401
    if (response.status === 401 && getRefreshToken()) {
      const newTokens = await refreshAccessToken();
      if (newTokens) {
        headers.Authorization = `Bearer ${newTokens.accessToken}`;
        response = await fetch(url, { ...options, headers });
      } else {
        removeTokens();
        window.location.href = '/login';
        throw new Error('Session expired');
      }
    }*/

    return response;
  };
};

// Функция обновления токена (использует baseURL Auth, так как endpoint
принадлежит Auth)
async function refreshAccessToken() {
  const refreshToken = getRefreshToken();
  if (!refreshToken) return null;

  const authBaseURL = process.env.REACT_APP_AUTH_API_URL ||
'http://localhost:5000';
  try {
    const response = await
fetch(`${authBaseURL}/api/auth/refresh?token=${refreshToken}`, {
      method: 'POST',
    });
    if (response.ok) {
      const data = await response.json();
      setTokens(data.accessToken, data.refreshToken || refreshToken);
      return { accessToken: data.accessToken };
    } else {
      removeTokens();
      return null;
    }
  } catch {
```

```
    removeTokens();
    return null;
  }
}

// Создаём два экземпляра: для Auth и для Core
const authApiRequest = createApiRequest(process.env.REACT_APP_AUTH_API_URL ||
'http://localhost:5000');
const coreApiRequest = createApiRequest(process.env.REACT_APP_CORE_API_URL ||
'http://localhost:5001');
const creditApiRequest = createApiRequest(process.env.REACT_APP_CREDIT_API_URL
|| 'http://localhost:5002');

export {
  authApiRequest,
  coreApiRequest,
  creditApiRequest,
  setTokens,
  removeTokens,
  getAccessToken,
  getRefreshToken,
  getRoleFromToken,
  isRoleAllowed,
};
</file>

<file path="src/services/authService.js">
import { UserManager, WebStorageStateStore } from 'oidc-client-ts';
import config from '../config';

const userManager = new UserManager({
  ...config,
  userStore: new WebStorageStateStore({ store: window.localStorage })
});

export const getUser = async () => await userManager.getUser();

export const login = () => userManager.signinRedirect();

export const loginCallback = () => userManager.signinRedirectCallback();

export const logout = () => userManager.signoutRedirect();

export const renewToken = () => userManager.signinSilent();

export default userManager;
</file>

<file path="src/services/core.js">
// services/core.js
import { coreApiRequest } from './api';

export const getAccounts = async () => {
  const response = await coreApiRequest('/api/accounts/employee', { method:
```

```
'GET' });
  if (!response.ok) {
    throw new Error('Failed to fetch accounts');
  }
  return response.json(); // предполагаем, что возвращается массив объектов
AccountDto
};

export const getAccountById = async (id) => {
  const response = await coreApiRequest(`/api/accounts/employee/${id}`, {
method: 'GET' });
  if (!response.ok) {
    throw new Error('Failed to fetch account');
  }
  return response.json();
};

export const deleteAccount = async (id) => {
  const response = await coreApiRequest(`/api/accounts/employee/${id}`, {
method: 'DELETE' });
  if (!response.ok) {
    throw new Error('Failed to delete account');
  }
  return response.json();
};

export const deposit = async (id, { amount, description }) => {
    const body = {
        sourceId: null,
        sourceType: "RealWorld",
        targetId: id,
        targetType: "Account",
        amount,
        description
    }

  const response = await coreApiRequest(`/api/transactions`, { method: 'POST',
body: JSON.stringify(body)  });
  if (!response.ok) {
    throw new Error('Failed to deposit');
  }
  return response.json();
};

export const withdraw = async (id, { amount, description }) => {
    const body = {
        sourceId: id,
        sourceType: "Account",
        targetId: null,
        targetType: "RealWorld",
        amount,
        description
    }
```

```
  const response = await coreApiRequest(`/api/transactions`, { method: 'POST',
body: JSON.stringify(body)  });
  if (!response.ok) {
    throw new Error('Failed to deposit');
  }
  return response.json();
};

export const getAccountTransactions = async (accountId, from, to) => {
  let url = `/api/accounts/employee/${accountId}/transactions`;
  const params = new URLSearchParams();
  if (from) params.append('from', from.toISOString());
  if (to) params.append('to', to.toISOString());
  if (params.toString()) url += `?${params.toString()}`;

  const response = await coreApiRequest(url, { method: 'GET' });
  if (!response.ok) {
    throw new Error('Failed to fetch transactions');
  }
  return response.json();
};
</file>

<file path="src/services/credit.js">
// services/credit.js
import { creditApiRequest } from './api';

/**
 * Получить список всех тарифов
 * @returns {Promise<Array>} массив TariffDto
 */
export const getTariffs = async () => {
  const response = await creditApiRequest('/api/tariffs', { method: 'GET' });
  if (!response.ok) {
    const error = await response.json().catch(() => ({}));
    throw new Error(error.message || 'Не удалось загрузить тарифы');
  }
  return response.json();
};

/**
 * Создать новый тариф
 * @param {Object} data - поля: name, interestRate, maxAmount, maxTermDays
 * @returns {Promise<Object>} созданный TariffDto
 */
export const createTariff = async (data) => {
  const response = await creditApiRequest('/api/tariffs', {
    method: 'POST',
    body: JSON.stringify(data),
  });
  if (!response.ok) {
    const error = await response.json().catch(() => ({}));
    throw new Error(error.message || 'Не удалось создать тариф');
  }
```

```
  return response.json();
};

/**
 * Получить тариф по ID
 * @param {string} id
 * @returns {Promise<Object>} TariffDto
 */
export const getTariffById = async (id) => {
  const response = await creditApiRequest(`/api/tariffs/${id}`, { method: 'GET'
});
  if (!response.ok) {
    const error = await response.json().catch(() => ({}));
    throw new Error(error.message || 'Не удалось загрузить тариф');
  }
  return response.json();
};

/**
 * Обновить существующий тариф
 * @param {string} id
 * @param {Object} data - поля: name, interestRate, maxAmount, maxTermDays
 * @returns {Promise<Object>} обновлённый TariffDto
 */
export const updateTariff = async (id, data) => {
  const response = await creditApiRequest(`/api/tariffs/${id}`, {
    method: 'PUT',
    body: JSON.stringify(data),
  });
  if (!response.ok) {
    const error = await response.json().catch(() => ({}));
    throw new Error(error.message || 'Не удалось обновить тариф');
  }
  return response.json();
};

/**
 * Удалить тариф по ID
 * @param {string} id
 * @returns {Promise<void>}
 */
export const deleteTariff = async (id) => {
  const response = await creditApiRequest(`/api/tariffs/${id}`, { method:
'DELETE' });
  if (!response.ok) {
    const error = await response.json().catch(() => ({}));
    throw new Error(error.message || 'Не удалось удалить тариф');
  }
  // при успехе возвращается пустой ответ с кодом 200
};

/**
 * Получить список кредитов (для сотрудника)
 * @param {string} [userId] - опциональный фильтр по пользователю
```

```
 * @returns {Promise<Array>} массив CreditDto
 */
export const getCredits = async (userId) => {
  const url = userId ? `/api/credits?userId=${encodeURIComponent(userId)}` :
'/api/credits';
  const response = await creditApiRequest(url, { method: 'GET' });
  if (!response.ok) {
    const error = await response.json().catch(() => ({}));
    throw new Error(error.message || 'Не удалось загрузить кредиты');
  }
  return response.json();
};

/**
 * Получить кредит по ID
 * @param {string} id
 * @returns {Promise<Object>} CreditDto
 */
export const getCreditById = async (id) => {
  const response = await creditApiRequest(`/api/credits/${id}`, { method: 'GET'
});
  if (!response.ok) {
    const error = await response.json().catch(() => ({}));
    throw new Error(error.message || 'Не удалось загрузить кредит');
  }
  return response.json();
};

/**
 * Одобрить кредит
 * @param {string} id
 * @param {Object} data - { approvedAmount, comment }
 * @returns {Promise<Object>} обновлённый CreditDto
 */
export const approveCredit = async (id, data) => {
  const response = await creditApiRequest(`/api/credits/${id}/approve`, {
    method: 'PATCH',
    body: JSON.stringify(data),
  });
  if (!response.ok) {
    const error = await response.json().catch(() => ({}));
    throw new Error(error.message || 'Не удалось одобрить кредит');
  }
  return response.json();
};

/**
 * Отклонить кредит
 * @param {string} id
 * @param {Object} data - { reason }
 * @returns {Promise<Object>} обновлённый CreditDto
 */
export const rejectCredit = async (id, data) => {
  const response = await creditApiRequest(`/api/credits/${id}/reject`, {
```

```
    method: 'PATCH',
    body: JSON.stringify(data),
  });
  if (!response.ok) {
    const error = await response.json().catch(() => ({}));
    throw new Error(error.message || 'Не удалось отклонить кредит');
  }
  return response.json();
};

/**
 * Получить платежи по кредиту (опционально)
 * @param {string} creditId
 * @returns {Promise<Array>} массив PaymentDto
 */
export const getCreditPayments = async (creditId) => {
  const response = await creditApiRequest(`/api/credits/${creditId}/payments`, {
method: 'GET' });
  if (!response.ok) {
    const error = await response.json().catch(() => ({}));
    throw new Error(error.message || 'Не удалось загрузить платежи');
  }
  return response.json();
};
</file>

<file path="src/services/users.js">
// services/users.js
import { authApiRequest } from './api';

/**
 * Поиск пользователей по запросу
 * @param {string} query - строка поиска (email или имя)
 * @returns {Promise<Array>} массив пользователей
 */
export const searchUsers = async (query = '') => {
  const response = await
authApiRequest(`/api/users/search?query=${encodeURIComponent(query)}`, {
    method: 'GET',
  });
  if (!response.ok) {
    throw new Error('Не удалось загрузить пользователей');
  }
  return response.json();
};

/**
 * Получение пользователя по ID
 * @param {string} id - UUID пользователя
 * @returns {Promise<Object>} объект пользователя
 */
export const getUserById = async (id) => {
  const response = await authApiRequest(`/api/users/${id}`, { method: 'GET' });
  if (!response.ok) {
```

```
    throw new Error('Не удалось загрузить данные пользователя');
  }
  return response.json();
};

export const addUserRole = async (userId, role) => {
  const response = await
authApiRequest(`/api/users/${userId}/role?role=${encodeURIComponent(role)}`, {
    method: 'POST',
  });
  if (!response.ok) {
    const error = await response.json().catch(() => ({}));
    throw new Error(error.message || 'Не удалось добавить роль');
  }
  return response.json(); // предположительно возвращает что-то или просто ok
};

export const removeUserRole = async (userId, role) => {
  const response = await
authApiRequest(`/api/users/${userId}/role?role=${encodeURIComponent(role)}`, {
    method: 'DELETE',
  });
  if (!response.ok) {
    const error = await response.json().catch(() => ({}));
    throw new Error(error.message || 'Не удалось удалить роль');
  }
  return response.json();
};

/**
 * Заблокировать пользователя
 * @param {string} userId
 * @returns {Promise<Object>}
 */
export const blockUser = async (userId) => {
  const response = await authApiRequest(`/api/auth/block/${userId}`, {
    method: 'POST',
  });
  if (!response.ok) {
    const error = await response.json().catch(() => ({}));
    throw new Error(error.message || 'Не удалось заблокировать пользователя');
  }
  return response.json();
};

/**
 * Разблокировать пользователя
 * @param {string} userId
 * @returns {Promise<Object>}
 */
export const unblockUser = async (userId) => {
  const response = await authApiRequest(`/api/auth/unblock/${userId}`, {
    method: 'POST',
  });
```

```
  if (!response.ok) {
    const error = await response.json().catch(() => ({}));
    throw new Error(error.message || 'Не удалось разблокировать пользователя');
  }
  return response.json();
};

// (Опционально) функции для управления ролями можно добавить позже
// export const addUserRole = async (userId, role) => { ... }
// export const removeUserRole = async (userId, role) => { ... }
</file>

</files>
```